LLM-IDS — Codebase Overview & Study Guide

A complete reference for understanding, explaining, and defending this project.

---

1. Elevator Pitch

LLM-IDS is a network intrusion detection system that replaces traditional rule-based or statistical-ML detection with a **local LLM acting as the classifier**. Instead of hand-written thresholds ("more than 100 SYNs/sec = attack"), every network flow is summarized into a compact statistics object and handed to a locally-running language model (via Ollama, default `llama3.1:8b`), which returns a classification — **Benign**, **Suspicious**, or **Attack** — with a plain-English explanation.

Everything runs locally: no cloud API calls, no data leaving the machine. The system can capture live traffic, analyze offline `.pcap` files, generate synthetic attack traffic for demos, and answer natural-language questions about what it has seen — all through one Streamlit dashboard.

One-line answer if asked "what does this do?"

> "It sniffs network traffic, groups packets into flows, extracts statistics like packet rate and SYN/ACK balance, and asks a local LLM to classify each flow as Benign, Suspicious, or Attack — with the reasoning behind it, not just a label."

---

2. Architecture at a Glance

```
Network Interface → Scapy Sniffer → Packet Collection → Flow Generator
       → Feature Extraction (Statistics / Protocol Info / Flags)
       → Flow Summary → Local LLM Analyzer → Classification + Explanation
       → SQLite → Streamlit Dashboard
```

The one architectural fact worth remembering above all others: the detection pipeline and the dashboard are **fully decoupled** — they communicate *only* through `storage/flows.db`. The pipeline (`main.py`, or the dashboard's own Live Capture tab) writes; the dashboard reads. Neither knows the other exists. This means: - The dashboard can be rebuilt in a completely different technology without touching detection logic. - Multiple writers (two `main.py` instances, Live Capture, Simulate Attacks, Upload PCAP) can all write to the same database concurrently without coordinating with each other.

---

3. The Pipeline, End to End

Stage	Module	What happens

1. Capture	`sniffer/capture.py`	Scapy sniffs raw packets off a network interface (or reads them from a `.pcap` file via `pcap_reader.py`)
2. Flow tracking	`sniffer/flow_tracker.py`	Packets are grouped into **flows** keyed by the 5-tuple (src IP, dst IP, src port, dst port, protocol); both directions of one conversation map to the *same* flow
3. Feature extraction	`features/extractor.py`	A finished flow becomes a features dict: **statistics** (duration, packet/byte counts, rate), **protocol info** (IPs, ports, protocol), **flags** (SYN/ACK/FIN/RST/PSH/URG counts + a `syn_without_ack` heuristic)
4. Classification	`analyzer/llm_client.py` + `prompt_builder.py`	Features are formatted into a prompt and sent to Ollama; the response is parsed and validated against `{Benign, Suspicious, Attack}`
5. Storage	`storage/db.py`	The verdict + features are written to a SQLite table (`flow_results`)
6. Display	`dashboard/app.py`	A Streamlit app reads from the same database and presents it across four tabs

What "a flow is finished" actually means

A flow is considered done — ready for classification — under **two** conditions: - An explicit **FIN or RST** TCP flag is seen (`FlowTracker.add_packet` sets `flow.closed = True`). - The flow has been **idle longer than `FLOW_TIMEOUT_SECONDS`** (default 15s) — checked periodically (`pop_finished_flows()`).

There is a **third** condition that only applies when the packet source is exhausted (end of a `.pcap` file, or the user clicked Stop on a live capture): `pop_all_flows()` force-drains *every* tracked flow regardless of state. This exists because a flow that never got a FIN/RST and hasn't timed out yet (e.g. a SYN flood — which by definition never completes a handshake) would otherwise sit in the tracker forever and never get classified. **This was a real bug found and fixed during development** — see §7.

---

4. Module-by-Module Reference

`config.py`

Central settings: network interface (`None` = Scapy picks default), flow timeout, Ollama host/model/timeout, and `DB_PATH` — computed as an **absolute path** anchored to the project root (`Path(__file__).resolve().parent`), not a bare relative string. This matters: a relative path breaks the moment the process is launched from a different working directory (which happened during development when testing via a launcher).

`main.py`

The original CLI entry point. Starts a sniffer thread, then loops forever: every `EXPIRY_CHECK_INTERVAL` seconds, pop finished flows, classify each, save, print. On **Ctrl+C**, it also flushes any flows still active at that moment (same `pop_all_flows()` reasoning as above) before exiting — otherwise whatever was mid-flight when you hit Ctrl+C would just be lost.

`sniffer/flow_tracker.py`

The heart of flow management. Key design point: **bidirectional keying**. `_make_key()` canonicalizes a 5-tuple so that packets from *either* direction of one conversation map to the same dictionary key — it compares `(src, dst, sport, dport, proto)` against its reverse and picks whichever sorts first. This is why a flow object's `.src_ip`/`.dst_ip` don't necessarily match the *first* packet's actual source — they reflect the canonical key, not literal direction. `FlowTracker` is thread-safe via a single `threading.Lock` protecting the internal dict.

`sniffer/capture.py`

Wraps Scapy in two modes: - **Blocking** (`start()`) — used by `main.py`, runs forever in its own thread. - **Async** (`start_async()`/`stop_async()`) — used by the dashboard's Live Capture tab, via Scapy's `AsyncSniffer`. Critically, `start_async()` **waits synchronously** for either a `started_callback` to fire or an exception to appear, so a permission failure (not running as Administrator/root) is raised immediately to the caller instead of leaving a silently-dead background thread. `sniff_error` is a property that also surfaces a **mid-capture** failure (e.g. the adapter gets unplugged) — this is polled live by `LiveCaptureSession`, not just checked once at stop time. - `list_interfaces()` returns friendly labels for the adapter-picker dropdown, passing the actual Scapy interface *object* (not a name string) to sidestep Windows's `\Device\NPF_{GUID}` formatting.

`sniffer/pcap_reader.py`

Same pipeline as live capture, but the packet source is `rdpcap()` reading an uploaded file instead of a live interface. At end-of-file it calls `pop_all_flows()` (not `pop_finished_flows()`) — the whole file is processed in milliseconds of wall-clock time, so a timeout-based close condition would essentially never fire naturally.

`sniffer/live_session.py` — `LiveCaptureSession`

The class backing the dashboard's Live Capture tab. Wraps sniffing + classification into a start/stop-able object that survives Streamlit's rerun-on-every-interaction model (stored in `st.session_state`, not a local variable). Key mechanics: - `start()` — starts the sniffer, then a background **classify loop** thread that periodically pops finished flows and classifies them. - `stop()` — signals the loop to stop and joins it (fast: the loop uses `Event.wait(timeout=...)` instead of `time.sleep()`, so it wakes immediately rather than finishing out its current sleep interval), then hands off whatever flows were *still active* to a **separate background flush thread** — because classifying dozens of leftover flows is dozens of real LLM calls, and `stop()` must return instantly regardless of how many there are or how slow the model is. `flush_running`/`flush_done`/`flush_total` let the UI show progress on that background work. - `self.results` — an in-memory, session-scoped list of everything this specific capture run has classified. The dashboard reads this directly instead of querying the database, so the table only ever shows *this session's* data, never history or another process's captures. Protected by a lock (`_results_lock`) since it's written

from background threads and read from the Streamlit UI thread; `results_snapshot()` returns a safe copy.

`features/extractor.py`

Turns a `Flow` into the features dict sent to the LLM. One subtlety worth knowing: **packets-per-second is only computed once a flow has run for at least 50ms** (`MIN_DURATION_FOR_RATE`). A single-packet flow has near-zero duration, and dividing by it would report an absurd rate (1000+ pps) for something as mundane as one DNS query — which the LLM's own prompt explicitly treats as attack evidence. Below that threshold, pps is reported as `0.0` rather than an inflated number.

`analyzer/prompt_builder.py`

Defines the system prompt for classification: a fixed schema (`classification`, `confidence`, `explanation`), explicit priority order (Attack > Suspicious > Benign), and concrete examples for each tier (e.g. Attack: "high SYN count with few or no ACKs"; Suspicious: "traffic to uncommon or known backdoor ports").

`analyzer/llm_client.py` — `LLMClient`

The single point of contact with Ollama. Three methods, all built on one private `_call()`: - `classify(features)` — the core classification call. **Fails safe to `Suspicious`**, never `Benign`, on any transport error, malformed JSON, or an out-of-schema classification value. The reasoning, stated directly in the code: *"an IDS that goes quiet on error is worse than one that over-flags."* - `generate_json(prompt)` — generic structured extraction (used by the Ask feature to turn a question into filters). Fails safe to `{}`. - `generate_text(prompt, fallback)` — free-form prose (incident reports, NL answers). Fails safe to a caller-supplied fallback string.

`analyzer/query_parser.py`

Powers the "Ask" tab. Two stages, one LLM call each: 1. `interpret(question, llm)` → `(filters, is_flow_question)` — a single call that **both** decides whether the question is actually about the flow data (vs. small talk like "how are you?") **and** extracts whatever filters it can (classification, protocol, IPs, port, relative time range, result limit). The relevance flag exists because early testing showed the system would otherwise answer *any* question — including greetings — by stitching real flow data onto a generic reply. 2. `summarize(question, results, llm)` — turns the matched rows into a plain-English answer. Two things make this work well instead of rambling: **pre-computed aggregate statistics** (classification/protocol counts, top ports, top source IPs) are handed to the model directly rather than making it count dozens of JSON records itself, and the prompt explicitly forbids describing the JSON schema or suggesting analysis techniques — both were observed failure modes with the smaller local model.

Filters are validated field-by-field (`_validate_filters`) before ever reaching the database — only whitelisted keys with the right types survive; nothing free-form gets through.

`analyzer/report_generator.py`

Expands one stored verdict into a full Markdown incident report (Summary / Evidence / Potential Impact / Recommended Remediation) via `generate_text()`. Falls back to a minimal report built from the stored classification/explanation if the LLM call fails, so a broken model never blocks the feature entirely.

`storage/db.py`

SQLite access layer — the *only* place SQL is ever written. Two tables: `flow_results` (every classified flow) and `feedback` (analyst corrections, foreign-keyed to `flow_results.id`). Key points: - `init_db()` sets `PRAGMA`

`journal_mode=WAL` — because this project routinely has several concurrent writers against the same file (multiple `main.py` processes, Live Capture's classify loop *and* its background flush thread, Simulate Attacks), plus the dashboard reading at the same time. WAL lets readers and writers proceed without blocking each other on every statement. - `query_results(filters, limit)` builds a parameterized query from a filters dict. **Every value is bound as a SQL parameter — nothing from `filters` is ever string-concatenated into the query.** This matters specifically because some filter values can originate from LLM output (the Ask feature): the LLM only ever supplies *values*, never SQL syntax, so there's no injection surface even though an LLM is "in the loop." - `save_result()` returns the new row's `id`, so callers (Live Capture) can reference that exact flow later (e.g. attaching feedback) without a lookup.

`dashboard/app.py`

The Streamlit UI, four tabs:

Tab	What it does
Live Capture	Pick an adapter, Start/Stop capture — runs entirely inside the Streamlit process via `LiveCaptureSession`. Needs admin/root. Below the controls: a results table and "Flow tools" panel (search, incident report, analyst feedback) scoped strictly to *this session's* captures — never history, never another process's data.
Upload PCAP	Analyze an uploaded capture file through the same pipeline as live capture.
Simulate Attacks	Generate synthetic traffic (SYN flood, port scan, benign browsing) with Scapy — packets are only ever *constructed*, never sent on a real interface — and run it through the real pipeline. No admin/root needed. Useful for demos without live malicious traffic.
Ask	Natural-language Q&A over stored history via `query_parser`.

`simulator/traffic_generator.py`

Builds three synthetic scenarios using only RFC 5737 reserved test-net addresses (so anything generated is obviously fake, never a real host): - **SYN Flood** — ~200 SYNs from one fixed 5-tuple, no ACKs. Deliberately collapses into **one flow** with an extreme packet count — because this IDS classifies flow-by-flow, a flood spread across randomized source IPs (like a real one) would be invisible to it; this scenario demonstrates what a per-flow detector *can* actually catch. - **Port Scan** — one source IP sweeping ~200 destination ports. This *necessarily* produces one flow per port (a 5-tuple can't span multiple ports), which means the scan pattern only becomes visible *across* flows sharing a source IP — exactly what per-flow classification structurally can't see on its own. The dashboard groups results by source IP afterward specifically to make this limitation, and the aggregate pattern, visible. - **Benign Web Browsing** — a handful of normal, balanced request/response exchanges — a contrast case that should classify as Benign.

---

5. Feature Tour (What Each Tab Actually Does)

- **Live Capture**: real packet capture, entirely in-browser control, session-scoped results table, per-flow incident reports and feedback.
- **Upload PCAP**: offline analysis of any Wireshark/tcpdump-compatible capture.
- **Simulate Attacks**: one-click synthetic traffic generation + classification, no network access required.
- **Ask**: natural-language querying of historical results, backed by a constrained, injection-safe filter schema — not a general-purpose SQL agent.

---

6. Design Decisions Worth Being Able to Defend

Q: Why fail-safe to "Suspicious" instead of "Benign" on an LLM error? An IDS that silently drops or downgrades flows when something goes wrong is more dangerous than one that over-flags. A false "Benign" hides a real problem; a false "Suspicious" just costs an analyst a few seconds of review.

Q: Why does the flow key not always match packet direction? Because both directions of one TCP/UDP conversation need to be treated as *one* flow, not two — otherwise a request and its response would be analyzed as unrelated one-way traffic, and features like "balanced request/response" would be meaningless. Canonicalizing the key (sorting the tuple) is the simplest way to guarantee that regardless of which side's packet arrives first.

Q: Why decouple the pipeline and dashboard through a database instead of, say, a shared in-memory object or a socket? It means either half can be restarted, replaced, or rebuilt independently. It also naturally supports multiple simultaneous writers (which this project actually has in practice) without any of them needing to know about each other.

Q: Why can't the LLM see multiple flows at once for something like a port scan? Architectural choice, not an oversight — the classifier judges one flow at a time by design, which keeps prompts small, judgments explainable per-flow, and the system fast. The real limitation this creates (scans/floods that spread across many flows are invisible to any single classification call) is handled by aggregating *after* classification (the source-IP grouping view) rather than trying to solve it inside the LLM call itself.

Q: How do you stop a slow LLM from blocking the UI? Two mechanisms: (1) Live Capture's Stop button hands remaining work to a background thread rather than blocking on it — the button always returns instantly regardless of how many flows or how slow the model is; (2) the classify loop itself uses an `Event.wait()` instead of `sleep()` so it reacts to a stop signal immediately instead of finishing out a wait interval first.

Q: How do you keep an LLM-powered "ask a question" feature from being a SQL injection risk? The LLM never produces SQL. It produces a small JSON object with specific, whitelisted keys (classification, protocol, IPs, port, time range). Each value is separately validated by type before being bound as a parameter into a query whose *structure* is entirely fixed in Python code. The LLM supplies data, never syntax.

Q: What happens if the LLM says something nonsensical, like an unrecognized classification label? It's coerced to `Suspicious` (same fail-safe philosophy) and the explanation is annotated to say the original label was unrecognized, so it's visible in review rather than silently substituted.

Q: Why SQLite instead of a "real" database? Zero setup, single file, easy to inspect/back up/reset, and — with WAL mode enabled — handles this project's actual concurrency needs (a handful of processes/threads, not a web-scale number of simultaneous writers) perfectly well.

---

7. Real Bugs Found and Fixed During Development (good defense material)

These demonstrate the system was actually tested under realistic conditions, not just written and left alone:

1. **Flows silently dropped at end-of-file.** `pop_finished_flows()` only returns flows that are closed or timed out *relative to wall-clock time*. Processing an uploaded `.pcap` file takes milliseconds, so a flow with no FIN/RST (like a SYN flood — the exact traffic this project cares about) would never satisfy either condition and would vanish from the results. Fixed by adding `pop_all_flows()`, used specifically at end-of-stream.
2. **PPS inflation on short flows.** Dividing by a near-zero duration made single-packet flows (e.g. one DNS query) appear to have 1000+ packets/second, which the LLM's own prompt treats as attack evidence — a source of false positives. Fixed with a minimum-duration floor before computing a rate at all.
3. **Live Capture's Stop button took several seconds, then silently dropped whatever was still in progress.** Two separate bugs: the background loop used `sleep()` instead of an interruptible `wait()`, and nothing flushed flows that hadn't closed/timed out yet when Stop was clicked. Fixed by making the wait interruptible and adding a background flush step (see `LiveCaptureSession.stop()` above).
4. **The "Ask" feature would answer any question, including "how are you?", with a fabricated data-flavored reply**, because the filter-extraction step always produced *some* query (falling back to "most recent 50 rows, unfiltered") regardless of whether the question had anything to do with flow data. Fixed by adding an explicit relevance classification (`is_flow_question`) that short-circuits the whole query/summarize pipeline for off-topic input.
5. **A relative `DB_PATH`** (`"storage/flows.db"`) broke the moment the app was launched from a different working directory. Fixed by anchoring it to the project root via `Path(__file__).resolve().parent`.
6. **A sniff-thread failure mid-capture (e.g. adapter unplugged) was silently discarded** by a bare `except: pass` — the user would just see "Packets seen" stop climbing with zero explanation. Fixed by capturing the exception and surfacing it through the same error-display mechanism as any other failure.

---

8. Known Limitations (be upfront about these if asked)

- **Per-flow classification is structurally blind to patterns that only exist across many flows** (e.g. a port scan touching hundreds of ports). Mitigated, not solved, by post-hoc source-IP grouping in the UI.
- **LLM filter extraction for the Ask feature is not perfectly deterministic** — the same question can occasionally extract slightly different filters between runs, since it's itself an LLM call.
- **Classification is sequential, not parallelized** — flows are classified one at a time, so throughput is bounded by how fast the local model responds (roughly 4–5 seconds per flow with `llama3.1:8b` on typical hardware observed during testing). A busy capture can take a while to fully process.
- **Progress bars in Simulate Attacks / Upload PCAP only reflect packet-parsing progress, not classification progress** — the UI can look "stuck" during the (potentially slow) classification phase even though it's working correctly.
- **A running Live Capture session has no automatic cleanup if the browser tab is closed without clicking Stop** — the background thread is a daemon thread, so it dies with the process, but nothing proactively stops it.

---

9. Testing Strategy

227 tests, all mocking the LLM client (a `FakeLLMClient`/`FakeLLM` stand-in) so the test suite needs no Ollama, no admin/root privileges, and no real network — it runs anywhere. Where it matters, tests use the *real* `FlowTracker`, `SQLite` (via a throwaway temp-file database per test), and real Scapy packet objects — only the actual network call and the LLM's response are faked. Coverage spans every module: flow tracking, feature extraction, the LLM client's fail-safe paths, the query parser's filter validation and relevance detection, the report generator, the traffic simulator, the database layer, live-capture session lifecycle (including thread-safety and timing-sensitive behavior like the fast-stop fix), and `main.py`'s orchestration.

Run with:

```
pytest tests/ -v
```

---

10. Quick-Reference Glossary

- **Flow** — one logical conversation between two endpoints, identified by the 5-tuple (src IP, dst IP, src port, dst port, protocol), covering both directions.
- **5-tuple** — the (src_ip, dst_ip, src_port, dst_port, protocol) key used to group packets into flows.
- **Fail-safe** — this project's term for "when something breaks, degrade toward the safer/more-visible outcome" (e.g. Suspicious instead of Benign; a fallback report instead of a crash).
- **WAL (Write-Ahead Logging)** — a SQLite journal mode that lets readers and writers operate without blocking each other on every statement.
- **`pop_finished_flows()` vs `pop_all_flows()`** — the former only returns flows that are closed or timed out (used during ongoing capture); the latter unconditionally drains everything (used when there's no more data coming, so there's no "later" to wait for).
- **`AsyncSniffer`** — Scapy's non-blocking capture class, used so the dashboard can start/stop capture from a button click instead of blocking the whole process.

Document generated as a reference for the LLM-IDS project. For setup and usage instructions, see `README.md`.